

Relazione 4

Federico Becchi

30-04-2026

1 Setup

Ho eseguito l'installazione del client VPN FORTINET senza alcun problema.

Come client SSH utilizzo la shell zsh di MAC OS

```
(base) federicobecchi@Federicos-MacBook-Air Downloads % echo $SHELL
/bin/zsh
```

2 Autenticazione tra tutti i nodi del cluster

Eseguo i seguenti comandi sul cluster.

Si creano una coppia di chiavi SSH con algoritmo RSA senza password, salvate in `~/.ssh/`

```
ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 0600 ~/.ssh/authorized_keys (cambio di permessi solo il proprietario puo' leggere
e scrivere)
```

```
User 1100 (4200)
4 r-Read
2 w-Write
1 x - Execute
4+2 = 6
Group 0
Other 0
```

Esito positivo.

3 Ambiente HPC (OS)

```
[federico.becchi@ui01 ~]$ cat /etc/system-release
CentOS Linux release 7.9.2009 (Core)
```

4 Relazione 1

I risultati per CPU e GPU si trovano nella relazione 1.

5 Relazione 2

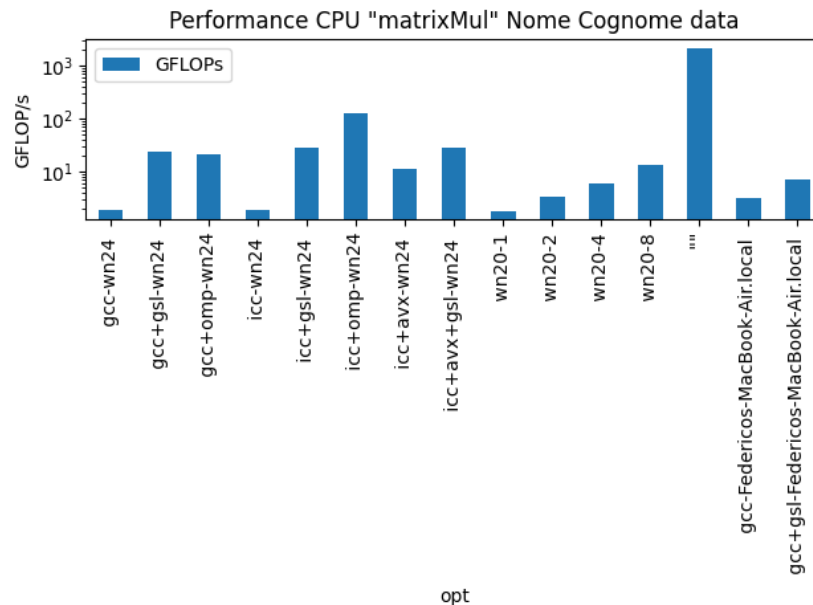


Figura 1: matrixMul.png

Considerazioni

Ci aspettano risultati migliori con le ottimizzazioni, la versione naive non utilizza nessuna parallelizzazione o istruzione SIMD (a parità di dimensione del problema). Inoltre con la GPU si ottengono risultati molto più alti rispetto alle singole CPU. Questo viene spiegato dal fatto che le GPU hanno migliaia di core (CUDA cores) e il problema di moltiplicazione matriciale è adattato a questa architettura poiché il calcolo della matrice risultante è indipendente dagli altri. Il confronto con la GPU nel PC locale non è possibile poiché non dispone di una GPU nvidia, è stato quindi necessario usare l'HPC d'ateneo.

6 Network

```
[federico.becchi@ui01 ~]$ sh band_lat.sh
```

Ho aggiunto queste due righe:

```
# SSH rifiuta la connessione (mpirun usa SSH per lanciare i demoni remoti)
# con "Host key verification failed."
docker compose exec wn1 sh -c 'mkdir -p ~/.ssh && ssh-keyscan -H wn1 wn2 >> ~/.ssh/
  known_hosts 2>/dev/null'
docker compose exec wn2 sh -c 'mkdir -p ~/.ssh && ssh-keyscan -H wn1 wn2 >> ~/.ssh/
  known_hosts 2>/dev/null'
```

Registrazione delle chiavi ssh nei rispettivi `known_hosts` altrimenti `mpirun` fallisce.
Esegue un test di bandwidth attraverso `mpirun`: intra-nodo e inter-nodo.

Risultati PC

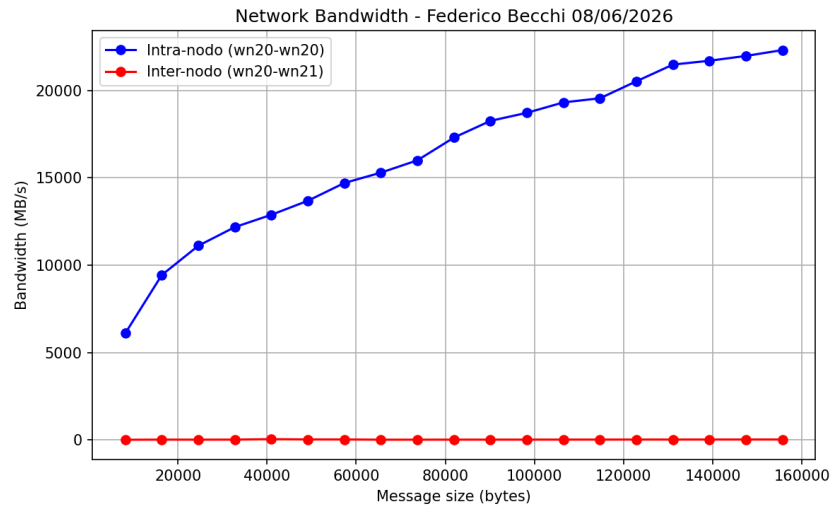


Figura 2: Network/PC/band.png

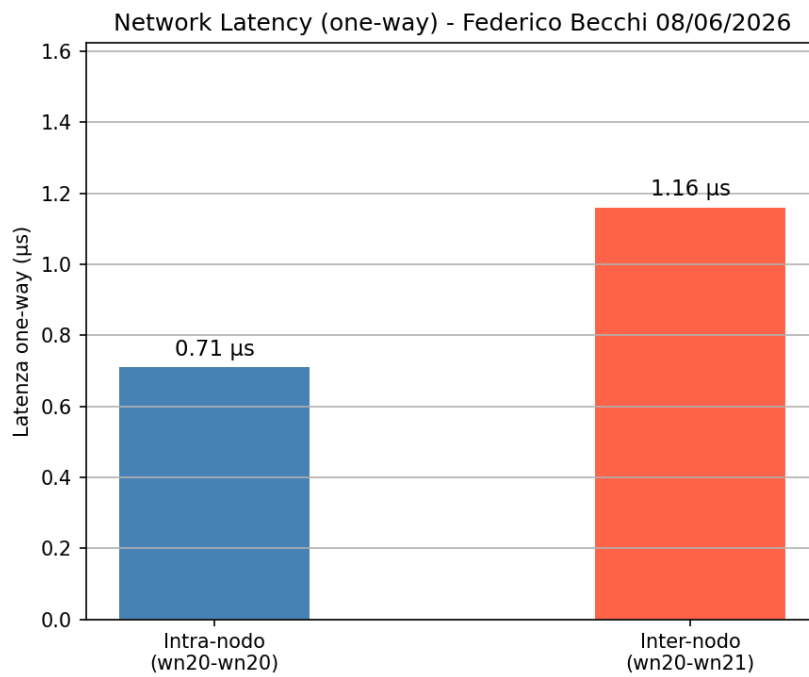


Figura 3: Network/PC/lat.png

Risultati Cluster

Viene eseguito `band_lat.slurm` sul cluster:

```
sbatch band_lat.slurm
```

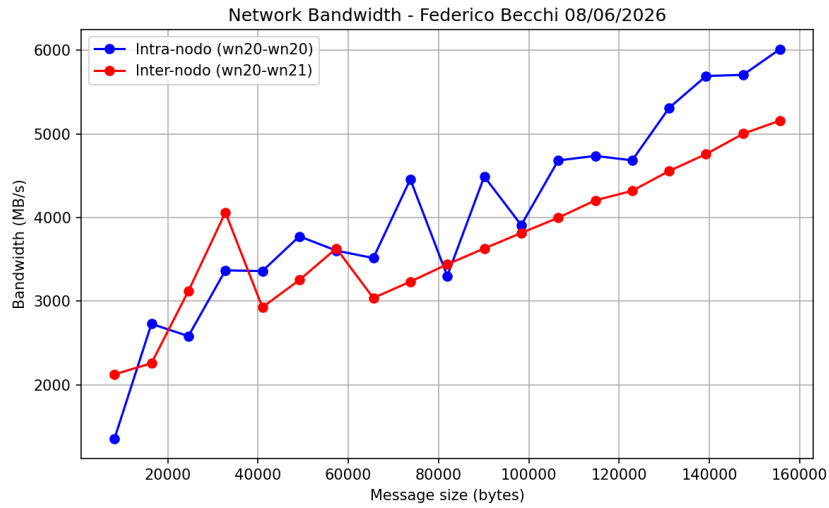


Figura 4: Network/Cluster/band.png

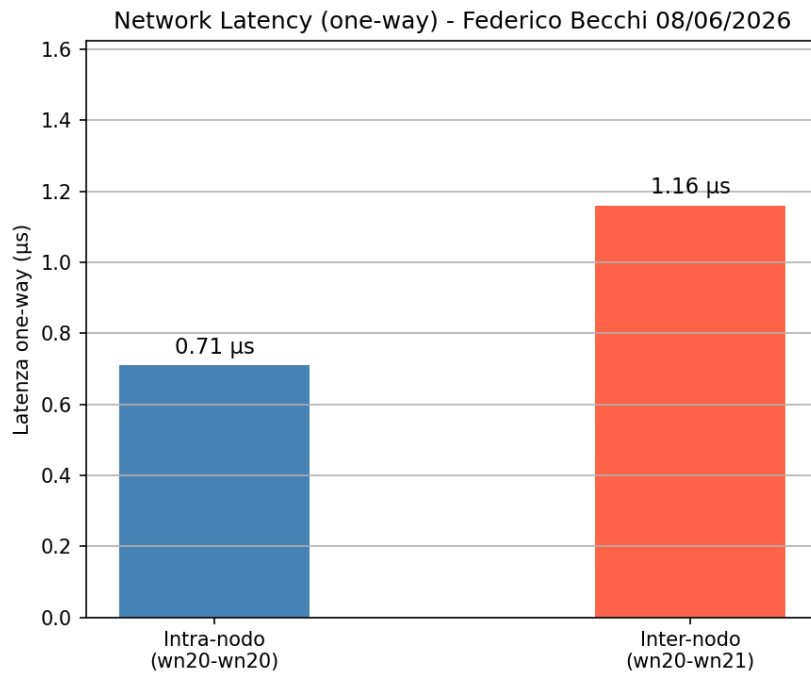


Figura 5: Network/Cluster/lat.png

Considerazioni

Sul cluster i test inter-nodo vengono eseguiti attraverso Omnipath quindi ci si aspettano i risultati di latenza e larghezza di banda piu' veloci rispetto al PC, in cui la rete cluster viene simulata attraverso due container docker. Rispettivamente i test di banda e latenza sono equivalenti.